Twitter Clone — Answer Key

Model answers to the code-defense questions

Advanced Programming — Dr. Saeed Reza Kheradpisheh

These are reference answers for the 30 questions. They describe the intended reasoning; a student's phrasing may differ, but a strong answer should hit the same key points (the *why*, not just the mechanics).

1. Architecture & Networking

Q1. Why a bounded thread pool instead of a thread-per-client, and what happens when it is exhausted?

Model answer. A fixed pool caps the number of live OS threads (20), bounding memory (each thread has its own stack) and context-switching overhead, and it prevents an unbounded number of connections from exhausting the machine. `newFixedThreadPool` is backed by an unbounded task queue, so when all 20 threads are busy a newly accepted connection's task simply *waits in the queue* until a thread frees up. Thread-per-client would spawn an unlimited number of threads and can crash the JVM under load; the trade-off is that a slow/blocked client holds a pooled thread, so the pool size must be chosen sensibly.

Q2. What does the line-delimited protocol assume, and how is the newline hazard avoided?

Model answer. It assumes every message is exactly one physical line with no embedded newline, because both sides frame messages with `readLine()`. If a raw `0x0A` appeared inside a tweet, `readLine()` would split one message into two, and the second half would fail to parse and desynchronize the stream. This is avoided because Gson's default (non-pretty) output is single-line and escapes any newline inside a string value as the two characters `\n`, so a literal newline byte never occurs inside the JSON.

Q3. What problem does the `requestId` solve versus matching by packet type?

Model answer. It correlates each response to the exact request that produced it. Matching by `type` breaks when two requests of the same type are in flight (e.g. two searches or two profile fetches): the first response would trigger whichever callback is registered for that type, producing a mismatch. A unique per-request UUID (echoed by the server) removes that ambiguity, and pushes—which carry no `requestId`—are routed through a separate path.

Q4. Why is `Dispatcher.dispatch` wrapped in `try/catch (RuntimeException)` inside `ClientHandler`?

Model answer. Handlers and DAOs can throw unchecked exceptions (NPE, `IllegalArgument`, a Gson parse error on a malformed field, etc.). Without the catch, the exception would propagate out of `handleLine` into the read loop, ending the loop and running the `finally` block that closes the socket—so a single bad request would disconnect the client. The catch turns it into a graceful `ERROR` response and keeps the connection alive.

Q5. Trace a tweet from user A to user B's feed. Who receives the push?

Model answer. A sends `CREATE_TWEET`; the Dispatcher validates the token and calls `TweetHandler.handleCreat` which inserts the tweet, media and hashtags in one transaction, then builds a `NEW_TWEET_PUSH` carrying the serialized tweet and calls `ConnectionRegistry.broadcastToFollowers(authorId, packet)`. That method sends the push to the author *and* to each of A's followers who is currently online (present in the `ONLINE_USERS` map), obtained from `FollowDAO.getFollowerIds`. B, if online and following A, receives it; the client routes the push to `HomePage`'s listener, which prepends the card. Only the author plus online followers receive it—not everyone—and replies are not broadcast (they notify the parent author instead).

Q6. How does the client tell a response from a push?

Model answer. `NetworkService.listenLoop` inspects `getRequestId()`. If it is non-null and a callback is registered under it, the message is a response and that callback runs. Otherwise it is an unsolicited push and is dispatched to the listeners registered for `packet.getType()`. Requests set a UUID that the server echoes, while `Packet.push(...)` leaves `requestId` null—so its presence or absence cleanly separates the two.

2. Concurrency & Thread Safety

Q7. Why `ConcurrentHashMap` in `Authenticator` and `ConnectionRegistry`?

Model answer. Both maps are static and shared by every client thread, which read/write them concurrently during login, logout, and broadcast. A plain `HashMap` under concurrent writes can corrupt its internal buckets during resize (historically an infinite loop), lose updates, or throw `ConcurrentModificationException` while `broadcast` iterates it. `ConcurrentHashMap` provides thread-safe, lock-striped access and safe iteration.

Q8. Why is `sendPacket synchronized`, and what race does it prevent?

Model answer. Two threads can write to the same client's socket: the client's own handler thread writing a response, and another user's thread pushing a message (e.g. a `LIKE_PUSH`) to this client via `ConnectionRegistry`. Without synchronization, the two `println` calls could interleave and merge two JSON objects onto one line, so the receiver's `readLine()` would get malformed JSON. `synchronized` makes each packet an atomic, whole-line write.

Q9. Why does the client listener use `Platform.runLater`?

Model answer. JavaFX allows the scene graph to be modified only on the JavaFX Application Thread. The listener runs on a separate network thread, so it marshals every callback back onto the FX thread with `Platform.runLater`. Touching the UI from the listener thread would throw `IllegalStateException: Not on FX application thread`.

Q10. Why a HikariCP pool instead of a fresh connection per DAO call, and does `close()` really close the socket?

Model answer. Opening a JDBC connection (TCP handshake, authentication, session setup) costs tens of milliseconds; a pool keeps a small set of live connections and hands them out, giving low latency and bounding the total number of DB connections under concurrency. With try-with-resources, calling `close()` on a pooled (proxied) connection does *not* close the physical socket to Postgres—it returns the connection to the pool for reuse—so each DAO method can safely "close" without tearing anything down.

3. Database, DAO & SQL

Q11. Show how parameterization prevents injection in the `ILIKE` search.

Model answer. The query is `WHERE username ILIKE ?` with `stmt.setString(1, "%"+query+"%")`. The `?` placeholder is transmitted to Postgres separately from the SQL text and is bound as data, never spliced into the statement string. So input such as `'; DROP TABLE users; -` is matched literally as a string and can never change the query's structure. String concatenation into the SQL would allow exactly that break-out.

Q12. Why must the three inserts in `createTweet` be one transaction?

Model answer. The tweet row, its media rows, and its hashtag links form one logical "post" and must all succeed or all fail. Without a transaction, a failure after the tweet was already committed would leave an inconsistent state—a tweet with missing images or hashtags, or partially linked tags. `setAutoCommit(false)` + `commit` makes it atomic, and `rollback` on `SQLException` undoes any partial work.

Q13. Explain `r`, `d`, and why `COALESCE` flattens retweets correctly.

Model answer. `r` is the feed row being scanned (a normal tweet or a repost); `d` is the tweet to display. `COALESCE(r.retweet_of, r.id)` returns `retweet_of` (the original's id) when `r` is a repost, and falls back to `r.id` when it is a normal tweet (`retweet_of` is NULL). Joining `tweets d` on that value yields the original for a repost and the tweet itself otherwise, while `r` still supplies who reposted and the ordering timestamp.

Q14. Why compute `liked/retweeted` on the server in one query?

Model answer. Computing them with correlated `EXISTS` subqueries parameterized by the viewer returns fully-rendered cards in a single round trip. Returning raw tweets and asking "did I like tweet X?" per tweet would be an N+1 of network round trips over the socket—slow and chatty—and would scatter the "truth" about state across the client.

Q15. What does `ON CONFLICT DO NOTHING` give follow/like, and why does it matter?

Model answer. It makes the write idempotent: repeating an existing follow/like is a harmless no-op instead of a primary-key violation. This matters for a double-click or a retry race, which would otherwise throw a duplicate-key `SQLException`. It also reports zero rows affected when the row already existed, which the handler uses to avoid creating duplicate notifications.

Q16. Justify the composite primary key on `follows/likes`.

Model answer. The natural identity of a follow or like *is* the pair of ids. A composite PK `(follower_id, following_id)` / `(user_id, tweet_id)` enforces "at most once" at the schema level, provides a covering index for lookups, and needs no extra column. A surrogate `SERIAL id` would still require a separate `UNIQUE` constraint to prevent duplicates, so it only adds storage and complexity here.

Q17. Why the recursive CTE for delete, and why is a single `DELETE` of parents and children safe?

Model answer. Replies form a tree through `parent_tweet_id`, so deleting a tweet must also remove its entire descendant subtree (and their likes); `WITH RECURSIVE` collects all of those ids. Parents and children can be removed in one `DELETE ... WHERE id = ANY(?)` because the `parent_tweet_id` foreign key uses the default `NO ACTION`, whose check is deferred to the *end of the statement*—by then every row in the set is gone, so no dangling reference exists. (`RESTRICT` would check immediately and fail.) Likes are deleted first because that table's FK has no cascade.

Q18. What does "idempotent additive schema" mean, and why re-run it every boot?

Model answer. Idempotent means running the script any number of times yields the same final state and never errors, achieved with `IF NOT EXISTS` / `ADD COLUMN IF NOT EXISTS`. Re-running on each startup is safe because existing objects are skipped and no data is dropped, while a brand-new database gets fully created—so nobody has to remember a manual migration step.

Q19. What N+1 problem does `getForTweets` with `ANY(?)` avoid?

Model answer. It loads media/hashtags for *all* tweet ids in one query using `WHERE tweet_id = ANY(?)` with a `java.sql.Array`, then attaches them from an in-memory map. Querying inside the row-mapping loop would issue one query per tweet —the N+1 pattern (1 feed query + N child queries)—multiplying DB round trips. Batching turns N+1 into 1+2.

Q20. Where is the BCrypt salt stored, why BCrypt over SHA-256, and why do equal passwords hash differently?

Model answer. `gensalt()` creates a random salt that BCrypt embeds *inside* the resulting hash string (the `$2a$` format), so no separate salt column is needed and `checkpw` re-reads it. BCrypt is preferable to plain SHA-256 because it is deliberately slow with a tunable work factor (resisting brute-force/GPU attacks) and is salted by default (defeating rainbow tables). Two users with the same password get different hashes because each receives a different random salt.

4. Protocol & Serialization

Q21. What error did the original `JsonObject` payload field cause, and how does `JsonElement` fix it?

Model answer. When a message arrived with `"payload": null`, Gson tried to bind a `JsonNull` to a `JsonObject`-typed field and threw `JsonSyntaxException: Expected a JsonObject but was JsonNull`, which `handleLine` treated as "Invalid JSON"—silently dropping legitimate null-payload messages. Typing the field as `JsonElement` lets it hold `JsonNull` cleanly; the getter converts `JsonNull` to `null` and otherwise returns the `JsonObject`, so all existing callers are unaffected.

Q22. What does `transient` do to `passwordHash`, and why does it matter?

Model answer. Gson skips `transient` fields when serializing. So even if a `User` object still carries its BCrypt hash after a DB read, that hash is never written into the JSON sent to a client—it cannot leak. It is defense-in-depth alongside not populating the field in client-facing responses.

Q23. Why the `request/ok/error/push` factory methods?

Model answer. Each factory encapsulates the correct field combination for one message kind (`ok` sets `status="OK"` and no token; `error` sets `status="ERROR"` plus a message; `request` sets type, token, payload). They keep call sites concise and prevent malformed packets (such as forgetting to set the status). `request` is the one that attaches a fresh `requestId`.

Q24. Why enforce the character limit on both client and server?

Model answer. The client check (the live counter and disabled button) gives instant feedback and avoids a wasted round trip, but the client is untrusted—a modified client or a raw socket could send an over-length tweet. The server is the source of truth, so `handleCreateTweet` re-checks the shared limit (and media count/size) and rejects violations. Client validation is UX; server validation is correctness and security.

5. Object-Oriented Design & Patterns

Q25. Name and justify the pattern behind (a) the singletons, (b) the DAOs, (c) the Dispatcher, (d) the registry + push listeners.

Model answer. (a) **Singleton**—one shared instance guarding a shared resource or state (the DB pool, the single client socket, the session context). (b) **DAO**—each class encapsulates all SQL for one table, isolating persistence from business logic. (c) **Front Controller / Dispatcher (command routing)**—a single entry point that routes each request type to its handler and applies cross-cutting authentication. (d) **Observer / publish–subscribe**—`ConnectionRegistry` publishes events to subscribed online clients, and on the client, listeners subscribe by packet type, decoupling event producers from consumers.

Q26. Advantages of centralizing authentication in the Dispatcher?

Model answer. Validating the token once in the Dispatcher means no handler can forget the check (no accidental unauthenticated access); the `userId` is resolved once and passed in, so handlers cannot disagree on identity; the public-vs-protected policy lives in one readable place; and there is no duplicated auth code. Per-handler checks would be repetitive and easy to get wrong.

Q27. Which principle does the `Binder` functional interface illustrate, and what would the code look like without it?

Model answer. It illustrates polymorphism / a lightweight Strategy via a lambda: the varying step (binding the WHERE/LIMIT parameters at indexes 3+) is abstracted so a single `query()` method handles viewer-parameter binding, execution, row mapping, and media/hashtag attachment for every read. Without it, that same connection/execute/map/ attach block would be copy-pasted across roughly seven read methods (feed, personalized feed, user tweets, by-id, replies, search, hashtag), inviting divergence and bugs.

6. JavaFX Client

Q28. Explain the optimistic like and how a later `LIKE_PUSH` is reconciled.

Model answer. The card updates the heart and count immediately for a responsive feel, sends `LIKE_TWEET`, and reverts if the server returns an error—so correctness is preserved. When a `LIKE_PUSH` later arrives (another user liked the same tweet), `FeedList.applyLikePush` finds the card by tweet id and calls `applyLikeUpdate(likeCount)`, overwriting the count with the server's authoritative value, keeping every client in sync. The push carries only the count, so another user's action does not flip *this* viewer's own liked state.

Q29. Why toggle a `dark` style class with CSS variables instead of inline styles?

Model answer. Toggling one class on the scene root flips the whole tree instantly, because every control inherits colors through CSS custom properties (`-fx-bg`, `-fx-text`, ...) that are redefined under `.root.dark`, and all colors live in one stylesheet. Inline styles would require imperatively walking every control on each toggle (and remembering each one), which is verbose, error-prone, and would not apply to nodes created later.

Q30. Discuss the trade-offs of Base64-in-DB media, and why cap the encoded length?

Model answer. Pros: everything lives in one place—images are stored atomically and transactionally with the tweet, need no separate file store or static server, and travel trivially inside the existing JSON socket protocol. Cons: Base64 inflates size by roughly a third, bloating the database and every feed query that returns media, and TEXT blobs are far heavier than a short path or URL; storing files on disk / object storage and keeping only a URL keeps rows small and lets images stream and cache independently. Because the encoded string travels inside a single-line JSON packet and lands in a TEXT column, the code caps the encoded length (`MAX_MEDIA_BASE64_LENGTH`) to bound packet and row size and prevent abuse.

Grading note: award full credit when the student explains the reason *(safety, concurrency, performance, UX) and can point to the relevant lines, even if the wording differs from the above.*